LANGGRAPH

Conditional Workflows

Video 7 -- Comprehensive Visual Notes

Detail	Value
Topic	Conditional Workflows -- Dynamic Branching / Routing
Core Idea	if/else logic inside graphs via Conditional Edges
Key API	graph.add_conditional_edges(source, router_fn, mapping)
Case Study 1	Quadratic Equation Solver (pure logic, no LLM)
Case Study 2	Customer Support Agent (LLM semantic routing)
Prerequisites	Video 5 (Sequential) + Video 6 (Parallel)

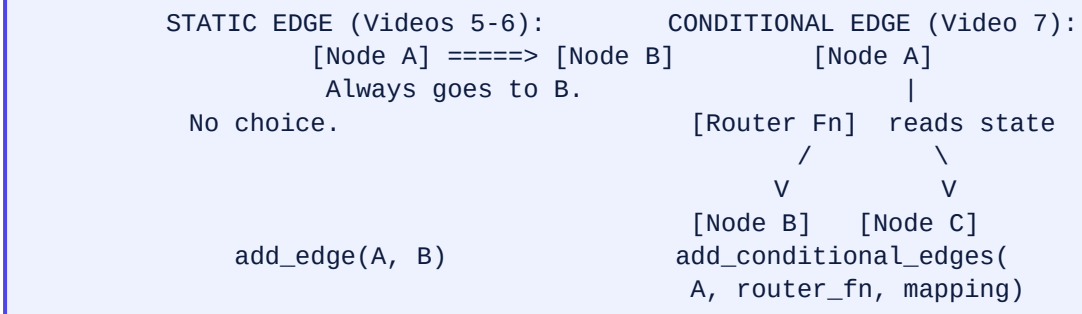
>> What You Will Learn

How conditional edges replace static edges for dynamic routing. The 3 components: Source Node, Routing Function, Mapping Dictionary. Two case studies: pure-math branching and LLM semantic routing. Extra: routing patterns, Pydantic for safe routing, common pitfalls, and full code templates.

1. Core Concept: What is a Conditional Workflow?

A conditional workflow introduces **dynamic decision-making**. Instead of a fixed path (A -> B always), the workflow **analyzes the current state** and **chooses the next step** based on logic. This is the graph equivalent of `if/else` in standard programming.

Static Edge vs. Conditional Edge



Analogy: Static edges are like a straight highway with no exits. Conditional edges are like a roundabout -- you enter, check the signs (state), and take the correct exit. Only ONE exit is taken per execution.

Edge Type	Code	Behavior	Use When
Static	add_edge(A, B)	Always A -> B	Fixed pipelines
Conditional	add_conditional_edges(A, fn, map)	A -> B or C (depends)	Branching, classification
Fan-Out (V6)	add_edge(START, B); add_edge(START, C)	A -> B AND C (parallel)	Independent tasks

! Conditional vs. Parallel -- The Key Difference

Conditional (V7): Only ONE branch executes. The router PICKS one path. Parallel (V6): ALL branches execute simultaneously. Think of it as: Conditional = OR (this or that). Parallel = AND (this and that). You can combine both in the same graph!

2. The 3 Components of a Conditional Edge

Every conditional edge has exactly three parts. Master these and you can build any branching logic.

A

The Source Node

The node WHERE the decision is made. After this node finishes, the routing function runs to decide what happens next. Examples: "calc_discriminant", "analyze_sentiment".

B

The Routing Function (The "Brain")

A Python function that reads the State and returns a STRING -- the name of the chosen path. This string is looked up in the Mapping dictionary. Signature: accepts State, returns str.

```
# Routing Function Signature:
# Input:  the current State dict
# Output: a STRING (the path name)

def decide_mood(state: CustomerState) -> str:
    if state["sentiment"] == "POSITIVE":
        return "happy_path"       # <-- path key (string)
    else:
        return "sad_path"         # <-- path key (string)
```

C

The Mapping Dictionary (Route -> Node Name)

Maps the STRING returned by the router to an actual node name in the graph. Decouples routing logic from graph structure. Optional but strongly recommended.

```
# Mapping: { router_output_string: actual_node_name }
path_map = {
    "happy_path": "generate_thanks",    # route -> node
    "sad_path":   "generate_apology",   # route -> node
}
```

All 3 Components Together

```
graph.add_conditional_edges(
    "analyze_sentiment",     # A: Source Node
    decide_mood,             # B: Routing Function
    {                         # C: Mapping Dictionary
        "happy_path": "generate_thanks",
        "sad_path":   "generate_apology",
    }
)
```

Execution Flow Through the 3 Components

```
Step 1: [Source Node] finishes executing
                                |
Step 2: [Router Function] reads state["sentiment"]
                  Returns: "happy_path"
                                |
   Step 3: [Mapping Dict] looks up "happy_path"
                Finds: "generate_thanks"
                                |
         Step 4: [generate_thanks] node executes
The other branch ("generate_apology") is SKIPPED entirely.
```

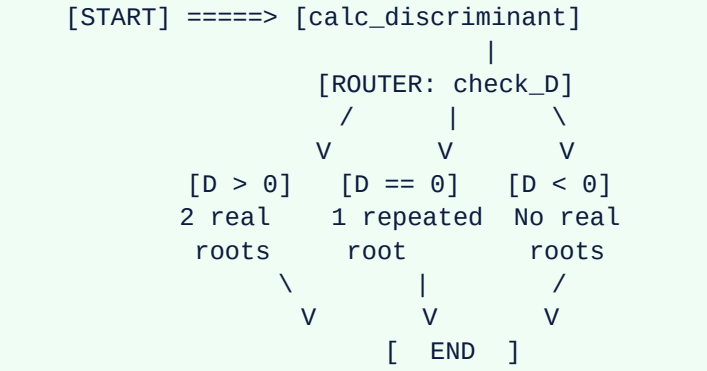
! Extra: Why Use a Mapping Instead of Returning Node Names?

(1) Rename nodes without touching the router function. (2) All possible routes documented in one place. (3) LangGraph validates mapping values exist as nodes at compile time. (4) Router logic stays semantic ('happy_path') not structural ('generate_thanks').

3. Case Study 1: Quadratic Equation Solver

A **pure-logic branching** example (no LLM). The discriminant $D = b^2 - 4ac$ of the quadratic equation $ax^2 + bx + c = 0$ determines which of three paths the workflow takes.

Quadratic Solver -- 3-Way Conditional Branch



Only ONE of the 3 branches runs per execution!

State and Routing Function

```
class QuadState(TypedDict):
    a: float
    b: float
    c: float
    discriminant: float
    result: str

def calc_discriminant(state: QuadState) -> dict:
    D = state["b"]**2 - 4 * state["a"] * state["c"]
    return {"discriminant": D}

# ROUTING FUNCTION:
def check_discriminant(state: QuadState) -> str:
    D = state["discriminant"]
    if D > 0:
        return "two_roots"
    elif D == 0:
        return "one_root"
    else:
        return "no_roots"
```

Graph Wiring

```
graph = StateGraph(QuadState)
graph.add_node("calc_disc", calc_discriminant)
graph.add_node("two_roots", solve_two_roots)
graph.add_node("one_root", solve_one_root)
graph.add_node("no_roots", solve_no_roots)

graph.add_edge(START, "calc_disc")

# CONDITIONAL EDGE:
graph.add_conditional_edges(
    "calc_disc",
    check_discriminant,
    {"two_roots": "two_roots",
     "one_root":  "one_root",
     "no_roots":  "no_roots"}
)

graph.add_edge("two_roots", END)
graph.add_edge("one_root", END)
graph.add_edge("no_roots", END)
```

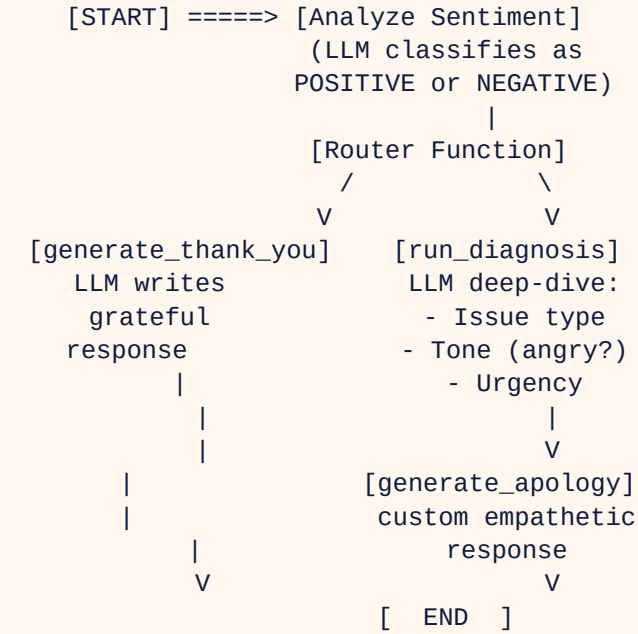
Extra: The Math Behind the Router

Condition	Meaning	Route	Example (a,b,c)
D > 0	2 distinct real roots	two_roots	1, -5, 6 => D=1
D = 0	1 repeated real root	one_root	1, -2, 1 => D=0
D < 0	No real roots (complex)	no_roots	1, 1, 1 => D=-3

4. Case Study 2: Customer Support Agent

This example introduces **Semantic Routing** -- where an **LLM determines the path** instead of hardcoded logic. The LLM reads natural language (a customer review) and classifies sentiment, which drives the routing decision.

Customer Support Agent -- Full Architecture



Two Types of Routing Compared

Aspect	Logic Routing (Case 1)	Semantic Routing (Case 2)
Decision maker	Python if/else	LLM classification
Analyzes	A number (D value)	Natural language text
Deterministic?	Yes, always	Mostly (LLM may vary)
Speed	Instant (no API)	Slower (needs LLM call)
Flexibility	Limited to coded rules	Handles ambiguous input
Error risk	None (pure math)	LLM format surprise

The Negative Branch Goes Deeper

>> Negative Branch: Deep Diagnosis

When routed to 'run_diagnosis', a specialized LLM performs deeper analysis: **Issue Type** (Bug, UI, Performance), **Tone** (Angry, Frustrated, Disappointed), and **Urgency** (High, Medium, Low). This rich diagnosis then flows to 'generate_apology' which crafts a custom, empathetic response. This shows how branches can contain multi-step sub-workflows after the decision point.

Implementation: Semantic Router with Pydantic

```
from pydantic import BaseModel, Field

class SentimentResult(BaseModel):
    sentiment: str = Field(
        description="Must be POSITIVE or NEGATIVE")

class SupportState(TypedDict):
    review: str
    sentiment: str
    diagnosis: str
    response: str

# Node: LLM classifies sentiment
def analyze_sentiment(state: SupportState) -> dict:
    structured_llm = llm.with_structured_output(SentimentResult)
    result = structured_llm.invoke(
        f"Classify POSITIVE or NEGATIVE:\n{state['review']}")
    return {"sentiment": result.sentiment}

# Router: reads classification from state
def route_by_sentiment(state: SupportState) -> str:
    if state["sentiment"] == "POSITIVE":
        return "positive"
    return "negative"    # Safe fallback

# Wiring:
graph.add_conditional_edges(
    "analyze_sentiment",
    route_by_sentiment,
    {"positive": "generate_thanks",
     "negative": "run_diagnosis"}
)
```

5. Critical: Structured Output for Safe Routing

When an LLM drives the routing decision (semantic routing), you **MUST** use Pydantic Structured Output. Without it, the routing is fragile and unreliable.

The Danger of Unstructured Routing

```
WITHOUT Pydantic:
LLM returns: "I believe the sentiment here is positive"
state["sentiment"] = "I believe the sentiment here is..."
Router: if state["sentiment"] == "POSITIVE"  -->  FALSE!
Takes WRONG branch or crashes with KeyError
WITH Pydantic:
LLM returns: {"sentiment": "POSITIVE"}
state["sentiment"] = "POSITIVE"
Router: if state["sentiment"] == "POSITIVE"  -->  TRUE!
Correct branch every time.
```

Without Pydantic	With Pydantic
LLM: "I think this is positive"	LLM: {"sentiment": "POSITIVE"}
String comparison fails	Exact match succeeds
Router takes wrong branch	Reliable routing every time
No validation	Guaranteed schema compliance
Debug nightmare	Clean, predictable flow

!! Extra: Defensive Routing Pattern

Even with Pydantic, add a fallback in your router. If the sentiment is neither 'POSITIVE' nor 'NEGATIVE', return a safe default like 'negative' (err on the side of caution). Normalize strings: use .upper() or .strip() before comparison. Always assume the LLM can surprise you -- defensive code prevents graph crashes.

```
# DEFENSIVE ROUTING FUNCTION:
def route_by_sentiment(state: SupportState) -> str:
    sentiment = state.get("sentiment", "").upper().strip()
    if sentiment == "POSITIVE":
        return "positive"
    return "negative"    # Default: err on side of caution
```

State Management Rule

Node Writes -> Router Reads -> Path Chosen

```
1. Input: state = {review: '...', sentiment: ''}
                                          (empty!)
        2. [analyze_sentiment] node runs
          LLM classifies, writes to state:
   state = {review: '...', sentiment: 'NEGATIVE'}
                                         ^^^^^^^^^^
3. [Router] reads state['sentiment'] = 'NEGATIVE'
                Returns: "negative"
    4. Mapping: 'negative' -> 'run_diagnosis'
Rule: Node BEFORE the router MUST populate the decision field!
```

6. Extra: Complete Code Template

A **complete, runnable** Customer Support Agent with conditional routing.

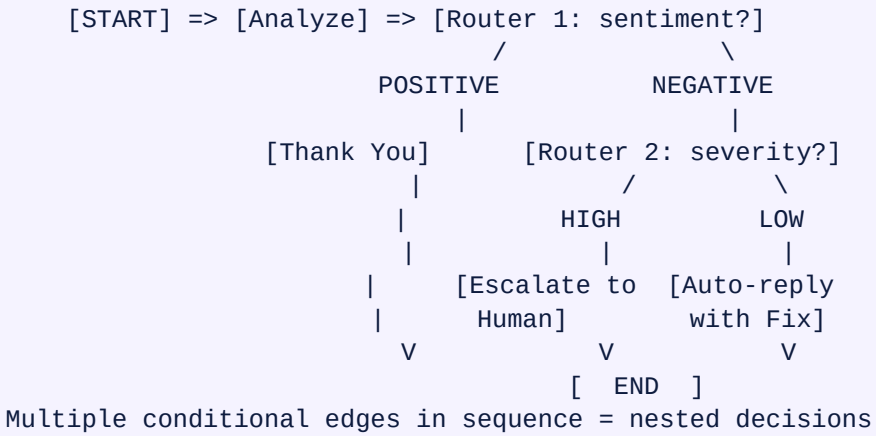
```
# === CUSTOMER SUPPORT AGENT (Part 1: Setup + Nodes) ===
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

class SentimentOutput(BaseModel):
    sentiment: str = Field(description="POSITIVE or NEGATIVE")

class SupportState(TypedDict):
    review: str
    sentiment: str
    diagnosis: str
    response: str

llm = ChatOpenAI(model='gpt-4o-mini')
s_llm = llm.with_structured_output(SentimentOutput)

def analyze_sentiment(state: SupportState) -> dict:
    r = s_llm.invoke(f"Classify: {state['review']}")
    return {"sentiment": r.sentiment}

def generate_thanks(state: SupportState) -> dict:
    r = llm.invoke(f"Write thank-you for: {state['review']}")
    return {"response": r.content}

def run_diagnosis(state: SupportState) -> dict:
    r = llm.invoke(f"Diagnose: {state['review']}")
    return {"diagnosis": r.content}

def generate_apology(state: SupportState) -> dict:
    r = llm.invoke(f"Empathetic reply for: {state['diagnosis']}")
    return {"response": r.content}
```

```
# === Part 2: Router + Graph Wiring ===
def route_sentiment(state: SupportState) -> str:
    return "pos" if state["sentiment"] == "POSITIVE" else "neg"

g = StateGraph(SupportState)
g.add_node("analyze", analyze_sentiment)
g.add_node("thanks", generate_thanks)
g.add_node("diagnose", run_diagnosis)
g.add_node("apology", generate_apology)

g.add_edge(START, "analyze")
g.add_conditional_edges("analyze", route_sentiment,
    {"pos": "thanks", "neg": "diagnose"})
g.add_edge("thanks", END)
g.add_edge("diagnose", "apology")
g.add_edge("apology", END)

app = g.compile()
result = app.invoke({"review": "Your product is terrible!",
    "sentiment":"","diagnosis":"","response":""})
print(result["response"])
```

7. Extra: Common Routing Patterns

Pattern	Description	Branches	Example
Binary Route	Simple if/else on one field	2 paths	Positive/Negative
Multi-Way Route	if/elif/else chain	3+ paths	D > 0 / = 0 / < 0
Semantic Route	LLM classifies with Pydantic	2+ paths	Customer sentiment
Threshold Route	Numeric comparison	2 paths	Score > 8 -> good
Chained Conditional	Multiple sequential routers	Nested	Type then severity
Default Fallback	else clause catches edge cases	N+1 paths	Unknown -> human

Extra Pattern: Chained Conditional Edges



>> Extra: Combining Conditional + Parallel

Real applications combine V6 + V7: Route positive reviews to a single thank-you node, but route negative reviews to a fan-out with 3 parallel diagnosis checks (bug, UI, performance), then fan-in to an aggregator that writes the apology. Each video's building block composes with the others!

Extra: Complete Edge Types Reference

Edge Type	Code	Behavior	Video
Normal	add_edge("a", "b")	Always A -> B	V5
Fan-Out	add_edge(START, "a"); add_edge(START, "b")	START -> A and B	V6
Fan-In	add_edge("a", "agg"); add_edge("b", "agg")	A,B -> Aggregator	V6
Conditional	add_conditional_edges("a", fn, map)	A -> B or C	V7
START/END	add_edge(START, "a"); add_edge("z", END)	Entry/exit points	V5

8. Extra: Common Pitfalls

Pitfall	What Goes Wrong	Fix
Router returns unmapped key	KeyError at runtime	All router returns must be in mapping
LLM returns free text	"I think positive" != "POSITIVE"	Use Pydantic structured output
No default else branch	Unhandled edge case crashes	Always have a fallback route
State missing decision field	Router reads None or empty	Prior node MUST write the field
Case-sensitive mismatch	"Positive" != "POSITIVE"	Normalize with .upper() in router
Missing branch -> END	Graph never terminates	Every terminal branch needs -> END
Router modifies state	Unpredictable side effects	Router should ONLY read, never write

!! Golden Rule: Routers are PURE Functions

A routing function should ONLY read state and return a string. It should NEVER modify state, call APIs, or have side effects. All work belongs in NODES. The router is a traffic cop -- it directs traffic, it does not drive.

9. Where Conditional Workflows Fit

Pattern	Video	Parallel?	Conditional?	Loops?	Complexity
Sequential	V5	No	No	No	* Easy
Parallel	V6	Yes	No	No	** Medium
Conditional (THIS)	V7	No	Yes	No	** Medium
Orchestrator-Worker	Later	Dynamic	Yes	No	*** Hard
Evaluator-Optimizer	Later	Optional	Yes	Yes	**** Expert

>> Your Toolkit So Far

V5: Static edges (A -> B always). V6: Fan-out/fan-in (A -> B AND C in parallel). V7: Conditional edges (A -> B OR C based on state). With these three building blocks, you can construct almost any graph topology. Remaining videos add cycles (loops) and dynamic orchestration on top of these fundamentals.

10. Quick Revision Checklist

Tag	Question
Concept	What is a conditional workflow? How does it differ from sequential?
3 Parts	Name the 3 components of add_conditional_edges().
Router	What does the routing function accept as input and return?
Mapping	Why use a mapping dict instead of returning node names directly?
Case 1	How does the quadratic solver decide between 3 branches?
Semantic	What is semantic routing? When do you need it?
Pydantic	Why is structured output critical for LLM-based routers?
Pure Fn	What is the golden rule for router functions?
vs Parallel	Conditional runs ___ branch; Parallel runs ___ branches.
Combine	Can you mix conditional + parallel in one graph? How?

Cheat Sheet: Conditional Workflow Patterns

What	Code
Conditional edge	graph.add_conditional_edges("src", router_fn, mapping)
Router function	def router(state: MyState) -> str: return "path"
Mapping dict	{"router_output": "actual_node_name", ...}
Pydantic for routing	llm.with_structured_output(PydanticModel)
Defensive fallback	return "default" as the else clause in router
Visualize graph	app.get_graph().draw_mermaid_png()
All branches to END	graph.add_edge("branch_node", END) # each one!

What's Next?

You now know three fundamental graph topologies: sequential (V5), parallel (V6), and conditional (V7). Upcoming videos combine these patterns with **loops** and **dynamic orchestration** to build truly autonomous agents. Make sure you can write the 3-component conditional edge from memory!